

Material de apoio:

Implementando o uso de texturas com OpenGL

Rossana B Queiroz



Iniciando...

- Função ou método que recebe o caminho do arquivo de textura e retorna o ID da textura
 - Primeiro, gera o ID de textura

```
// Gera o identificador da textura na memória  
glGenTextures(1, &texID);  
glBindTexture(GL_TEXTURE_2D, texID);
```

Iniciando...

- Depois, ajusta-se os parâmetros de *wrapping* e *filtering*

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

Iniciando...

- Agora fazemos o carregamento do arquivo de imagem, utilizando uma biblioteca auxiliar

```
int width, height, nrChannels;  
  
unsigned char *data = stbi_load(filename.c_str(), &width, &height,  
&nrChannels, 0);
```

OBS.: Estamos usando a biblioteca stb_image: <https://github.com/nothings/stb>
Podemos diretamente adicionar o arquivo .h e .cpp em nosso código

Iniciando...

- Por último, mandamos os dados para OpenGL armazenar a textura na memória

```
if (data)
{
    if (nrChannels == 3) //jpg, bmp
    {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
    }
    else //png
    {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, data);
    }
    glGenerateMipmap(GL_TEXTURE_2D); // geração do mipmap
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
```

Iniciando...

- Boas práticas: liberar a memória do buffer e desconectar o ID (“unbind”)

```
stbi_image_free(data);  
glBindTexture(GL_TEXTURE_2D, 0);
```

- Não precisa ser dentro do método, mas na inicialização é necessário ativar o buffer de textura da OpenGL:

```
glActiveTexture(GL_TEXTURE0);
```

Iniciando...

- Sobre as unidades de textura: devem ser habilitadas antes de conectá-las (bind) corretamente antes de fazer a chamada de desenho, por exemplo:

```
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, textID1);  
glActiveTexture(GL_TEXTURE1);  
glBindTexture(GL_TEXTURE_2D, textID2);  
  
glBindVertexArray(VAO);  
glDrawArrays(GL_TRIANGLES, 0, 3);
```

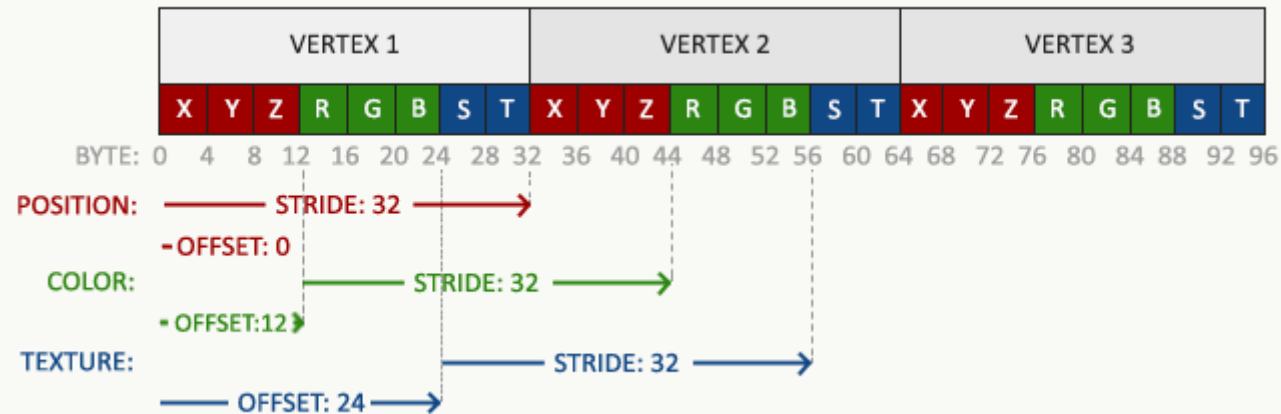
E os buffers?

- Temos agora mais um atributo para os vértices!

```
GLfloat vertices[] = {  
  //x y z r g b s t  
  -0.5 , -0.5 , 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, //v0  
  0.5 , -0.5 , 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, //v1  
  0.0 , 0.5 , 0.0, 0.0, 0.0, 1.0, 0.5, 1.0 //v2  
};
```


E os buffers?

- Temos agora mais um atributo para os vértices!



E os buffers?

- Temos agora mais um atributo para os vértices!

```
glGenVertexArrays(1, &VAO); glGenBuffers(1, &VBO); glGenBuffers(1, &EBO);

glBindVertexArray(VAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

// position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// color attribute
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
// texture coord attribute
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);
```

E no(s) shader(s)?

- Vertex shader

```
#version 450 core
layout (location = 0) in vec3 position;
layout (location = 1) in vec3 color;
layout (location = 2) in vec2 tex_coord;

out vec4 vertexColor;
out vec2 texCoord;

uniform mat4 model;
uniform mat4 projection;

void main()
{
    gl_Position = projection * model * vec4(position, 1.0f);
    vertexColor = vec4(color, 1.0);
    texCoord = vec2(tex_coord.x, tex_coord.y);
}
```

Atributo cor não é obrigatório, mas pode ser mantido pra criar efeitos

E no(s) shader(s)?

- Fragment Shader

Atributo cor não é obrigatório, mas pode ser mantido pra criar efeitos

```
#version 450 core
in vec3 vertexColor;
in vec2 texCoord;

out vec4 color;

// pixels da textura
uniform sampler2D tex_buffer;

void main()
{
    color = texture(tex_buffer, texCoord);
}
```

Nome do método ou função que você criou!

Finalizando...

- Não esquecer de chamar o método para carregar a imagem 😊

```
GLuint texID = generateTexture("../textures/wall.jpg");
```

- Não esquecer de especificar a variável uniform que vai conter os dados da textura no fragment shader

```
glUniform1i(glGetUniformLocation(shader.ID, "tex_buffer"), 0);
```

- Antes da drawcall, ativar o buffer de textura e fazer a conexão com o buffer desejado:

```
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, texID);
```

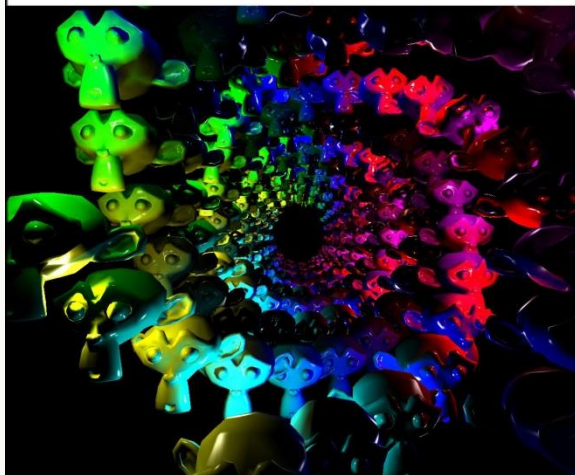
```
glBindVertexArray(VAO);  
glDrawArrays(GL_TRIANGLES, 0, 3);
```

Referências

- Akenine-Moller, T., Haines, E., & Hoffman, N. (2018). *Real-time rendering*. AK Peters/CRC Press.
- Slides sobre CG dos professores passados: Christian Hofsetz, Cristiano Franco, Marcelo Walter, Soraia Musse, Leandro Tonietto e Rafael Hocevar
- <https://learnopengl.com/Getting-started/Textures>

Referências

Anton's OpenGL 4 Tutorials



Anton Gerdelan

Ebook para Kindle

Muitos materiais online disponíveis em:

<http://antongerdelan.net/opengl/>